

WorldEpisode: Auditing Silent State Drift in Robot-Learning Datasets

Alba Tellez Fernandez
URDF Studio

Abstract

Robot-learning datasets can be locally valid and behaviorally wrong: every file loads, but a command is replayed at the moment it was issued rather than when the motor executed it, the same reconstructed scene appears in both training and testing, or an asset’s collision role no longer matches the state under evaluation. We call this *silent state drift*. This paper presents *WorldEpisode*, an executable state-integrity contract for robot-learning datasets: five machine-checkable invariants per episode (persistent identity, frame and clock mappings, representation roles, temporal events, provenance) plus the two corpus-level obligations that carry the headline results, lineage-disjoint evaluation splits and loss-explicit conversion.

On a public SO-101 LeRobot release, a random episode split leaks every tested world lineage and yields 0.850 offline success for a behavioral-cloning probe; a scene-disjoint split reduces measured leakage to zero (0.000) and the same probe to 0.000. Declaring the inferred four-frame, 133 ms actuation delay on a real trajectory reduces validation joint RMSE from 4.732 to 1.862 degrees and same-trace MuJoCo/Genesis replay RMSE from 3.425 to 1.563 degrees. A round trip over ten public LeRobot episodes preserves 1,935 state/action rows with maximum numerical error 0.0 while reporting source-absent semantics as explicit conversion loss, and the validator detects all 14 injected fault classes. The paper claims no universal file format, no simulator-independent physics, and no benchmark score inflation without policy reruns; ACT/Diffusion rollouts, famous-benchmark reruns, Isaac/SAPIEN execution, and external adoption remain explicit open gates.

1 Introduction

Imagine training a behavioral-cloning policy for three weeks on a public robot dataset. Every Parquet, video, and scene file passes its local loader and the simulator reports an 85% success rate; deployed on the real arm, the policy immediately misses the object. Nothing was corrupt: the dataset encoded joint values in a unit the controller did not declare, the replay script treated command timestamps as motor-effective timestamps, and the random split let the policy see the same reconstructed room in both training and evaluation. We call this failure class *silent state drift*. The same pattern appears outside robotics as well; Section 6 returns to that point as future work, since this paper’s measured evidence is entirely robotics. Table 1 makes the stakes concrete: each row is a file that still loads, the behavior it silently breaks, and the requirement that catches it. The rest of the paper turns each requirement into an executable rule and measures what happens when it is missing.

Requirement IDs follow `AREA.number`: `ACTION` is the action-channel contract (units, frame, command/effective timing), `TIME/FRAME` are clocks and spatial frames, `REP` is representation role, `ASSET` is asset identity and digest, `SPLIT` is evaluation-split lineage, `WORLD/TRACE` are the world revision and its episode deltas, `PROV/QUALITY` are provenance and uncertainty, and `DATASET` is

Table 1: Failure modes that pass local validation but break behavior, and the WorldEpisode requirement that catches each. Requirement IDs are defined in Section 2.

File still loads, but...	Silent consequence	Requirement
joint values use an undeclared unit	controller misreads the scale; policy misses the object	ACTION.001 (declare units)
command time is replayed as motor-effective time	replay and deployment drift off the real trajectory	ACTION.002 (declare timing)
a random split shares a reconstructed scene across train and test	success is inflated and does not transfer	SPLIT.001 (lineage-disjoint splits)
a real-to-sim export keeps appearance but drops the collision proxy	the policy collides with geometry it cannot see	REP.001 (declare asset role)
camera and lidar logs are fused across clock domains	spatial fusion error beyond tolerance	TIME.002 (declare clock drift)
an asset URI looks right but its content digest changed	the wrong geometry is used under evaluation	ASSET.002 (verify digest)

corpus scale. Section 2 defines all of them. Robot learning is the domain evaluated here because it is a deep stress test: real hardware, asynchronous control, multi-camera data, reconstructed worlds, simulator exports, and policy evaluation.

Existing formats solve important pieces of this chain. LeRobot vertically integrates robot learning from motor interfaces through dataset collection, policies, and deployment [3]; LeRobotDataset v3 stores high-frequency states, actions, and timestamps in Parquet, visual streams in MP4 shards, and episode indexing in metadata [8], and is the container this paper evaluates in depth with active experiments on real downloaded data. Rerun provides a physical-data layer for multimodal recordings, NCore documents explicit pose graphs and sensor conventions, and OpenUSD, SimReady, glTF, and simulator-native formats describe composed worlds and assets, with Gaussian splats entering standard containers through glTF’s `KHR_gaussian_splatting` extension [20, 21]; these are covered as lighter binding-retention pilots in Section 4. Recent real-to-sim systems such as RoboSnap and DROID-Sim show that Gaussian-splat environments can support trajectory replay, synthetic data generation, and policy evaluation at dataset scale [19]. That progress makes the missing contract more urgent rather than less: a layered scene can be visually faithful while dropping an object’s collision role, and a replayable trajectory is still wrong if the action vector is interpreted under a different controller contract at deployment time.

Related work. Documenting datasets is not new. Datasheets for Datasets [6] and model cards standardize prose descriptions of provenance and intended use; MLCommons Croissant [13] adds a machine-readable metadata layer for ML-ready datasets; and Open X-Embodiment [17] aggregates robot data under a shared schema. Evaluation leakage is a well-documented driver of the reproducibility crisis in ML-based science [9]. WorldEpisode differs in three ways. First, it binds these fields to an *immutable, content-addressed world revision* with closed identity, frame/clock, role, and provenance graphs, so cross-container agreement is checkable rather than merely described. Second, it is *executable and fail-closed*: a one-line preflight blocks a native LeRobot or Rerun artifact that cannot prove the contract, and every conversion emits a machine-readable loss report. Third, it treats *lineage-disjoint evaluation splits* as a first-class requirement with measured consequences, not as optional documentation. It is a contract and validator layered on existing containers, not a

replacement format or a dataset card.

The contract is designed to be neutral to storage container, asset representation, and runtime, and to make conversion loss explicit rather than silent. This paper claims that neutrality only as far as the evidence reaches: LeRobotDataset v3 in depth on real downloaded data, five further containers as binding-retention pilots, two runtimes (MuJoCo, Genesis) as tested same-trace replay adapters, and a loss report on every conversion. Where a mapping is designed but untested — Isaac Sim, SAPIEN, game engines, autonomous-driving logs — the paper says so instead of claiming it. The empirical claims are bounded accordingly: they demonstrate executable failure mechanisms and auditable conversion semantics, not a completed multi-lab benchmark or a real-robot deployment study.

Contributions.

1. **WorldEpisode**, an executable state-integrity contract for robot-learning datasets, expressed as five graphs of invariants (identity; frames and clocks; representation roles; temporal state and events; provenance and lineage), with a JSON Schema, a semantic validator, and a one-line fail-closed preflight for native LeRobot and Rerun artifacts.
2. **Loss-explicit binding**: round-trip conversions that emit machine-readable loss reports, with measured native-versus-sidecar semantic retention.
3. **Controlled failure studies**: scene-lineage leakage, action-timing replay drift, and real-to-sim contract drift, plus two non-robotics pilots — each an executable mechanism, with every stronger claim bounded by an explicit open reproduction gate.

Section 2 states the contract, Section 3 makes it executable, Section 4 reports what the audit finds, and Section 6 states the boundary of every claim, with the full evidence-boundary table and gate commands in the appendix.

2 The Contract: Five Graphs and Two Corpus Obligations

A robot-learning episode must answer a practical question: what must be known before it can be safely converted, replayed, synchronized, augmented, or used for evaluation? WorldEpisode answers it with five coupled graphs at the level of a single episode record, two corpus-level obligations that follow from those graphs, and one scale requirement for production datasets. Table 2 maps each question to the structure that answers it, the requirement IDs that make it checkable, and the case in Section 4 that tests it. WorldEpisode specializes these requirements for robot-learning episodes.

2.1 The five coupled graphs

Figure 1 shows the five graphs that make a spatial state replayable and auditable rather than just stored. Making them mutually referential is what turns a silent error into a detectable one: an identity, timing, role, event, or lineage error becomes a broken reference instead of a hidden assumption.

World revision and deltas (WORLD.001, TRACE.001). Each record references one immutable, content-addressed world revision plus an ordered list of deltas; any change to geometry, scale, calibration, collision proxies, physical parameters, or object identity creates a new revision. Content hashes let two datasets prove they refer to the same world without relying on filenames.

Table 2: The contract at a glance: each question a valid episode must answer, the graph or obligation that answers it, the executable requirement IDs, and the evidence case that tests it.

Question	Answered by	Requirements	Tested in
Which state revision am I in?	world revision + ordered deltas	WORLD.001, TRACE.001	Case 4
What entities does it contain?	identity graph	ENTITY.001	Case 4
Where and when is it valid?	frame + clock graphs	FRAME.001-2, TIME.001-2, ACTION.001-2	Cases 2, 4
What is each asset safe for?	representation-role graph	REP.001, ASSET.001-2	Cases 3, 4
What actually happened?	temporal-event graph	TRACE.001	Case 4
Where did this come from?	provenance graph	PROV.001, QUALITY.001	Case 4
What was lost in conversion?	obligation: loss-explicit conversion	CONVERT.001	Cases 5, 6
Can the evaluation split leak?	obligation: lineage-safe splits	SPLIT.001	Cases 1, 7
Can this scale past one folder?	dataset manifest + indexes	DATASET.001	App. C

Identity graph (ENTITY.001). Every physical or logical entity receives a persistent identifier, and the same `entity_id` must be used by segmentation masks, Gaussian groups, meshes, collision bodies, simulator actors, language references, and interaction events. If a language command says “pick up the mug,” the perception mask, the simulator object, and the collision proxy must identify the same mug.

Frame and clock graphs (FRAME.001, TIME.001, ACTION.002). Every spatial value declares its frames, units, transform direction, and validity interval, consistent with and extending ROS conventions [15]; every stream declares a clock domain, timestamps, synchronization mapping, and estimated drift, with the rigor NCore demonstrates for pose graphs [14]. Actions declare command time and effective motor time rather than assuming those are the same instant. This graph turns a suspicious replay from “the policy is bad” into a diagnosable offset, transform-direction, or unit error.

Representation-role graph (REP.001). One entity can have many representations (splat, mesh, point cloud, collision proxy, mass/inertia record), each declaring a role and a digest-verified asset descriptor. The role field is essential: a splat may be valid for appearance but invalid for collision, while a simplified proxy may be valid for collision but unsuitable for visual training.

Temporal-event graph (TRACE.001). Records store interaction events — contact, grasp, attachment, collision patch, reset, intervention, failure — each referencing entities, time, confidence, and source, turning raw coordinate streams into auditable milestones.

Provenance graph (PROV.001). Every derived asset identifies its source episodes, sensors, reconstruction algorithm and version, calibration revision, hashes, creator, license, and quality metrics, mapping to existing provenance vocabularies rather than inventing an incompatible one.

The full normative field lists are specified in the v0.1 RFC and its JSON Schemas; the paper keeps only what each graph must guarantee.

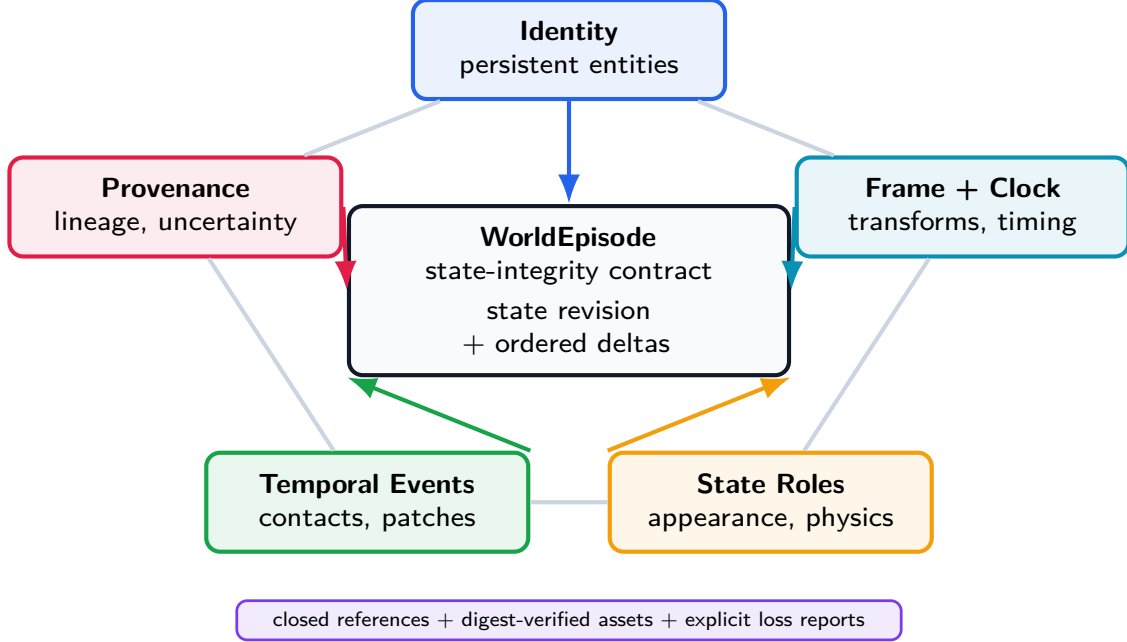


Figure 1: The five-graph model. WorldEpisode makes identity, frames/clocks, state roles, temporal events, and provenance mutually referential so conversion, synchronization, and replay errors become detectable instead of hidden.

2.2 Two corpus-level obligations, plus scale

Loss-explicit conversion (CONVERT.001). WorldEpisode reuses existing standards rather than replacing them. USD [1] carries composed simulation worlds, glTF carries Gaussian appearance assets [20], NCore carries capture and pose-graph conventions [14], Rerun carries physical-data recordings with column chunks [18], ROS/MCAP carries logs, and domain-native engines carry their own telemetry. A converter may externalize or approximate fields, but it must emit a machine-readable loss report instead of silently discarding semantics. The standard permits lossy conversion; it forbids silent lossy conversion.

Lineage-safe splits (SPLIT.001). Record-level random splits are not enough when multiple records share a reconstructed room, map patch, physical object, source capture, calibration, generated asset lineage, or state ancestry — exactly what the provenance graph records. A WorldEpisode split manifest can therefore enforce disjointness by constraints such as `world_lineage`, `physical_entity`, `source_capture`, `reconstruction_run`, or `generated_asset_family`.

Scale (DATASET.001). Large robot corpora require a dataset manifest above the episode records: namespaces, resolver priority, shard catalogs, materialized indexes, split catalogs, and append-only versions. This lets a reader locate all SO-101 pick-place traces for a world-disjoint split by index lookup rather than by crawling storage paths.

The initial WorldEpisode profile is intentionally narrow: rigid tabletop manipulation with fixed-base single- or dual-arm robots. That bounded domain keeps conformance executable rather than aspirational. Future profiles can extend this specification to humanoids, locomotion, deformables, fluids, multi-agent game worlds, and autonomous-driving logs.

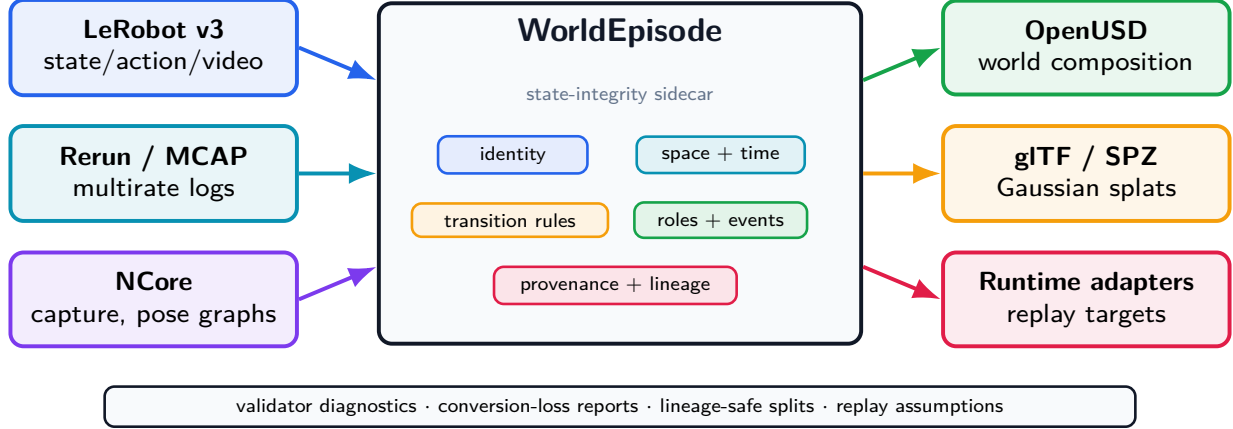


Figure 2: WorldEpisode is the semantic bridge between spatial data containers, state/world assets, and runtime targets.

3 The Sidecar Contract, Made Executable

WorldEpisode delivers the contract as a sidecar blueprint around existing formats. It does not ask a LeRobot dataset to become a USD scene, or a USD scene to become a policy-training table. Instead, it records the relationships those containers need in order to agree about the same state: the state revision, persistent entities, space/time conventions, transition contract, asset roles, interaction events, provenance, and split lineage. The sidecar is not a mandate to package a global robot corpus as millions of small folders: the single-episode record is the semantic unit, and production datasets add a manifest layer (namespaces, URI resolvers, shard catalogs, materialized indexes, split manifests, append-only snapshots) so that opening a corpus starts from the manifest and its indexes rather than from a recursive storage scan.

Figure 2 shows the intended data flow. Dataset, log, and telemetry containers plug in on the left; world assets and replay targets plug in on the right; the WorldEpisode sidecar holds the semantics that would otherwise be scattered across undocumented loader assumptions. This gives the paper its central rule: keep native containers for the data they are good at storing, but make the cross-container assumptions explicit, checkable, and loss-aware.

3.1 Runtime adapters

WorldEpisode is not tied to a single rendering or physics engine. A runtime becomes a trusted target by implementing an adapter that preserves the same semantic invariants and emits replay evidence, which keeps the integration direction clean: datasets are not rewritten around simulator-native assumptions; simulators compile their scene loading, action interfaces, and replay reports against the sidecar. The adapter contract has three layers:

- **Invariant interface:** ingests immutable world revisions, frame and clock graphs, entity identity, representation roles, action channels, asset digests, provenance, and quality records.
- **Asynchronous schema extension:** lets an engine register cloth, deformable, fluid, articulation, material, or procedural-scene extensions without weakening the core episode record.
- **Deterministic replay accountability:** records runtime identity, version, solver, timestep, actuator parameters, initialization state, command/effective timing, latency, tolerance envelopes, and measured drift.

The current implementation reflects this boundary across two codebases: WorldEpisode has tested minimal MuJoCo [7] and Genesis replay adapters for the same LeRobot control-loop trace, and URDF Studio, the companion simulator-transfer workbench, passes a shared `SimBackend` conformance suite for both engines with a one-episode MuJoCo–Genesis comparison report. The Isaac mapping is emitted but untested, and SAPIEN remains adapter-required. The claim is therefore not simulator-independent physics but runtime-neutral accountability: every simulator reports against the same state contract, so cross-runtime drift becomes measurable rather than hidden inside loader conventions.

3.2 The executable robotics profile

Complete action semantics. An action vector is not portable without its operational contract. Each action channel must declare actuator, control mode, parameterization, reference frame, units, absolute/delta/velocity semantics, limits, command rate, command timestamp, effective timestamp, latency model, interpolation, normalization, gripper semantics, and missing-value policy. A vector such as `[0.01, 0, 0, 0, 0, 0, 1]` is otherwise ambiguous: it could be a tool-frame delta, base-frame delta, velocity command, normalized policy output, or delayed target increment.

Implementation artifact. We implement the contract as four executable components: a JSON Schema for structural validity, a machine-readable conformance requirement file, an installable semantic validator, and a round-trip experiment runner. The validator maps each diagnostic to a stable requirement identifier such as `TIME.001`, `FRAME.002`, or `ASSET.002`. The experiment runner takes a WorldEpisode package, exports binding-native artifacts plus sidecars, reimports them, compares a versioned semantic projection, injects controlled faults, and writes reproducible result artifacts. The projection itself is published as `conformance/projections/uss-core-23.v0.json` and validated against `schemas/semantic-projection-v0.schema.json`. URDF Studio remains the initial implementation root, but the contract is represented independently in the public schema and validator.

One-line preflight. The validator is packaged as `worldepisode`. A blocking preflight can be run from the shell with `worldepisode preflight <dataset-or-manifest>` or from a training script with a single Python call such as `preflight_lerobot(dataset.root).raise_if_failed()`. Native LeRobot folders and Rerun `.rrd` recordings are recognized directly; without a WorldEpisode manifest or sidecar, they fail closed on missing world revision, entity identity, representation role, action timing, frame/clock, split-lineage, and conversion-loss controls. This makes the semantic audit a pre-training check rather than a separate post-hoc documentation exercise. The repository pairs the validator with converters, leakage and replay tools, a clean-room reader, and a runtime adapter report; Section 4 presents what each one measures.

4 Evidence: What the Audit Finds

The experiments are written as failure investigations rather than as isolated unit tests, and they run in three acts: what drift costs when it goes uncaught (Cases 1–3), what the contract catches and repairs (Cases 4–6), and how the same audit scales beyond one dataset (Case 7, with the dataset-scale audit in Appendix C). All measured evidence here is robotics; two deterministic non-robotics pilots are deferred to Section 6. Results that are executable but not yet measured are marked open inline; their commands, required artifacts, and acceptance rules are consolidated in Appendix B, and the boundary of every claim is tabulated in Appendix A.

4.1 What drift costs

Case 1: scene-lineage leakage in evaluation splits (SPLIT.001). The most dangerous failure is not a crash; it is a benchmark that looks successful for the wrong reason. We audit ArmnetBench v0.1’s single-arm SO-101 LeRobot release [2] by deriving eight WorldEpisode-style `world_lineage` hashes, one for each task-scene and camera-layout group. A standard random episode split trains on 320 episodes, tests on 80, and leaks all test scene lineages into training. The same Torch MLP state-action behavioral-cloning probe then reaches 0.850 offline imitation success under a fixed normalized-RMSE tolerance of 0.25.

When the split is made scene-disjoint, the apparent success disappears. The held-out eye-drop scene lineages have zero overlap with training, the test set contains 100 episodes, and offline BC success drops to 0.000. Mean episode normalized RMSE rises from 0.183 to 0.363, a $1.98\times$ increase. This is an offline action-imitation benchmark, not a physical rollout claim or a claim about state-of-the-art LeRobot policies. Its purpose is narrower and diagnostic: it exposes that a conventional split can reward memorizing a background world rather than learning a transferable manipulation behavior: the number moved with the split, not the policy. **HELP: C1**

The leakage penalty survives a second model class: a closed-form temporal ridge baseline with a three-frame state history, executed directly on the committed compact LeRobot state/action split packages, reaches 0.925 offline success on the random episode split and 0.420 on the scene-disjoint split, a 0.505 success-rate drop (mean episode normalized RMSE 0.157 to 0.255); this is still not ACT or Diffusion and not a rollout result. A generated memorization pilot with world/entity lineage labels shows the same shape: random splits reach 1.000 accuracy while world-disjoint and entity-disjoint splits fall to 0.500 and 0.562.

To keep the stronger claim from being implied before it is measured, the repository also emits an ACT/Diffusion gate from the same split manifest: train/test episode allowlists, four virtual split manifests with 27 digest-verified source files, four compact physical LeRobot state/action split packages totaling 241,470 rows, four LeRobot-native training jobs (`act` and `diffusion` on random and scene-disjoint splits), and a simulator or physical rollout contract. The gate is intentionally open in this release; no ACT, Diffusion Policy, IsaacLab, or hardware success number is claimed, and the compact packages omit video payloads until source videos are mirrored with committed digests. The command path and acceptance rule are stated in Appendix B.

Case 2: action-timing and replay drift (ACTION.002). LeRobot’s control stack can decouple policy inference from the fixed-rate low-level controller: commands may be queued, chunked, interpolated, or consumed after policy output [3]. On the public SO-101 trajectory, WorldEpisode records policy-output, enqueue, queue-consume, and motor-receive timestamp semantics, plus zero-order-hold interpolation and a constant-frame latency model.

Estimating the effective delay on the calibration prefix gives four 30 Hz frames, or 133 ms. On the validation suffix, naive command-time alignment has joint RMSE 4.732 degrees. Timestamp-aware alignment has RMSE 1.862 degrees, a $2.54\times$ improvement. We then execute the same WorldEpisode action contract through two minimal six-joint position-servo replay adapters. In MuJoCo, naive command-time replay has RMSE 3.425 degrees and timestamp-aware replay has 1.563 degrees, a $2.19\times$ improvement. Genesis same-trace replay gives the same measured profile: 3.425 degrees under naive command-time replay and 1.563 degrees under timestamp-aware replay, also a $2.19\times$ improvement. The two engines agree to roughly seven significant figures (MuJoCo 3.4247237 versus Genesis 3.4247236 naive) precisely because both integrate the same minimal six-joint position-servo model; we therefore report this as an adapter-conformance result — the same action contract executes consistently across runtimes — and explicitly not as evidence that the engines’ underlying

physics agree. The Isaac adapter is emitted as a ready mapping from the same contract, but is not tested in this environment. The failure avoided here is a common one: blaming a simulator or controller when the dataset never said when the command actually took effect.

A dependency-free replay-adapter conformance harness guards additional runtime adapters: on constant frame delay, zero-order hold for missing commands, and asynchronous queue selection, a naive scheduler fails while the contract-aware scheduler matches the expected trace with zero numerical error. Together with URDF Studio’s shared `SimBackend` conformance suite and its one-episode MuJoCo–Genesis comparison report, these results close the narrow same-trace second-runtime replay gate for the minimal joint replay profile; they do not claim contact-rich task replay, Isaac/SAPIEN execution, or simulator-independent behavior. [HELP: C5](#)

Case 3: real-to-sim contract drift (ACTION.001, REP.001). Recent Gaussian/OpenUSD real-to-sim systems correctly separate visual reconstruction from interactive physical assets. WorldEpisode targets the contract between those layers and the source robot episode. We therefore add two deterministic ablations. In the action-contract ablation, a policy succeeds in a simulator that interprets its output as absolute joint-radian targets, but the same vector fails under a deployment proxy whose controller expects delta-degree commands. In the representation-role ablation, an appearance-only Gaussian export drops the collision proxy; the straight-line policy succeeds in simulation and collides with the real foreground geometry under the deployment proxy. In both ablations the drifted contract has 2/2 simulated successes and 0/2 deployment successes, while the WorldEpisode contract recovers 2/2 deployment successes. This is a controlled proxy, not a RoboSnap or hardware rerun, but it shows why visual reconstruction is not enough without action contracts and representation roles: appearance alone does not make a scene replay-safe.

Figure 3 summarizes the measured changes for three experiment families where explicit state semantics alter the outcome.

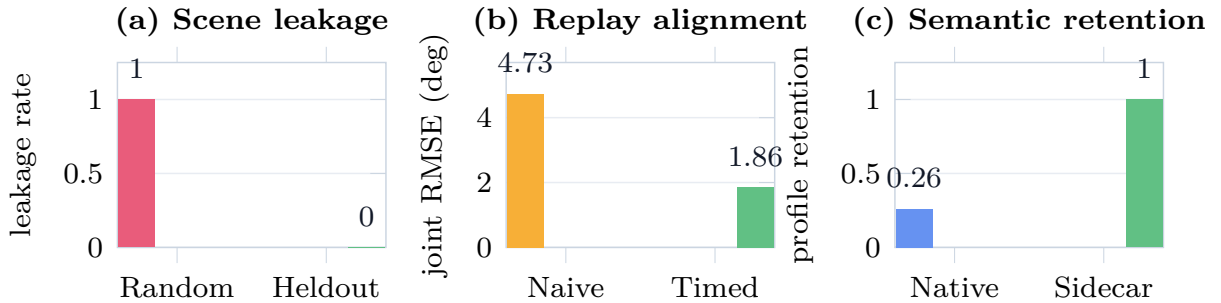


Figure 3: Quantitative summary of the three measured effects used in the evaluation. Lower is better for leakage and replay RMSE; higher is better for semantic retention.

4.2 What the contract catches

Case 4: validator fault detection (all requirement IDs). We inject 14 faults into the baseline package: missing clock domains, incomplete cross-clock mappings, unknown frames, missing transform intervals, unknown event entities, missing representation roles, digest errors, action-contract omissions, non-content-addressed worlds, missing delta sequences, missing provenance, and missing quality records. The validator detects all 14 expected requirement failures with 1.000 recall and 0.933 precision. Two hand-authored independent fixtures also trigger the expected diagnostics. The single false-positive requirement is useful rather than harmful: removing an action reference

frame triggers both `ACTION.001` and the downstream `FRAME.001` reference check. The same suite includes a unit sanity check from `lerobot/svla_so101_pickplace`: the first mirrored SO-101 frame has six joint values with maximum magnitude 98.924 in degrees and 1.727 after radian conversion, so treating those degree values as radians would violate the action/state unit contract even though the tensor shape is valid.

To move beyond synthetic mutations, the experiment runner also writes a pilot natural-source corpus from committed public LeRobot artifacts and source-level public benchmark metadata. Across `lerobot/svla_so101_pickplace`, `lerobot/pusht`, `armnet/armnetbench_v01_lerobot_so101`, `DROID`, and `BridgeData V2`, it records 19 observed cases: source-absent camera extrinsics, calibration transforms, action units, controller-latency models, a random split with measured scene-lineage leakage, and public evidence gaps from the famous-benchmark call-out audit. These are naturally occurring source omissions and evaluation risks, not claimed as maintainer-confirmed dataset bugs or measured score-inflation results. The runner emits dataset-specific diagnostic reports for all five datasets; the `DROID` and `BridgeData V2` reports are marked source-level-only until pinned `WorldEpisode` conversions and false-positive review are available. [HELP: C3](#)

The same validator is exposed as a pre-training command rather than only as an experiment script. The preflight regression covers four cases: a valid `WorldEpisode` manifest passes, an invalid `WorldEpisode` fixture fails, a native LeRobot v3 folder without a `WorldEpisode` sidecar fails closed, and a Rerun `.rrd` recording without a sidecar fails closed. This is the intended usage pattern for labs: run one blocking line before launching a multi-day policy training job. To reduce single-implementation risk, we also run an internal clean-room reader that does not import the `worldepisode` package. It parses the public schema, summarizes the minimal example, and catches all 18 expected requirement instances across 17 fixtures with 1.000 recall (the 14 injected-fault fixtures above plus one valid baseline and two independently authored fixtures; one independent fixture expects three requirement failures, so 17 fixtures carry 18 expected instances). This is not external adoption, but it verifies that the public artifacts are readable outside the reference SDK path.

Case 5: loss-explicit dataset conversion (`CONVERT.001`). To test an actual `LeRobotDataset` rather than only a capability model, we implement `LeRobotDataset v3` $\rightarrow$ `WorldEpisode` $\rightarrow$ `LeRobotDataset v3` on five-episode batches from two pinned public datasets: `lerobot/svla_so101_pickplace` [11] and `lerobot/pusht` [10]. The converter downloads the real Parquet meta-data and data shards, builds a schema-valid `WorldEpisode` manifest and sidecar, exports a compact `LeRobotDataset v3` package, reads it back, and asserts exact equality.

This is a two-dataset batch audit for active conversion, not a coverage claim for all LeRobot datasets. Across ten episodes, the round trip preserves 1,935 rows of source-native action tensors, 1,935 state rows, sample timestamps, frame indices, episode indices, global sample indices, task indices, and camera video timestamp ranges with maximum absolute error 0.0. It also preserves the derived physical-frame records through the sidecar. More importantly, it does not fake missing semantics: camera extrinsics, robot/world calibration, action units, and controller latency are reported as source-absent rather than guessed. This is the difference between a convenient conversion and an auditable one. [HELP: C7](#)

Case 6: binding retention across containers (`CONVERT.001`). The same executable artifacts measure how much of the semantic contract is preserved by native containers alone. Each binding writes a native payload and a `WorldEpisode` sidecar, then the experiment reimports both and checks exact equality against the versioned `uss-core-23` projection in `conformance/projections/`

`uss-core-23.v0.json`: episode identity/task outcome, world revision and asset descriptor, embodiment identity and URDF [16] asset, frames/transforms, clocks/mappings, entity identity/roles/assets, action control and timing contracts, trace binding and asset descriptor, events, ordered deltas, provenance, quality, split lineage, and replay assumptions. This is an executable pilot projection, not a universal score of each format’s value. Across seven pilot bindings, Table 3 shows that common containers preserve 17–39% of this projection natively, while WorldEpisode sidecars recover that projection for the tested dataset/log/world bindings. The sidecar column is by construction: because the sidecar serializes the projection fields directly, the 1.000 entries verify lossless round-trip of the contract, not an emergent property; the informative measurement is the native gap each container leaves. A Gaussian asset binding alone preserves 17% natively and 30% with limited sidecar metadata, confirming that an asset format is not an episode–world contract.

Table 3: Semantic field retention in the pilot binding model.

Binding	Native	With WorldEpisode
LeRobot v3	0.261	1.000
Rerun RRD	0.348	1.000
NCore	0.304	1.000
MCAP/ROS2	0.261	1.000
OpenUSD/SimReady	0.391	1.000
glTF Gaussian asset	0.174	0.304

4.3 Scaling the audit

Case 7: source-level audit of famous benchmarks (SPLIT.001). The ArmnetBench result is the measured leakage case. To scale the same diagnostic, we add a source-level call-out audit over five famous public robot-learning benchmarks: Open X-Embodiment/RT-X [17], DROID [5], BridgeData V2 [22], LIBERO [12], and CALVIN [4]. The audit asks whether public metadata exposes lineage splits, content-addressed world revisions, persistent entity identity, action timing contracts, frame/clock graphs, and replay or conversion-loss reports. All five benchmarks have at least one high-severity open control; the three real-robot datasets have five each. This is a call-out target list, not a claim that their published scores are inflated: a separate inflation-proof gate requires a benchmark-specific WorldEpisode conversion, lineage/timing audit, published-policy rerun, corrected evaluation, and measured score drop before any famous benchmark can be labeled inflated. A targeted DROID-100 LeRobot-subset rerun tool executes that path over 100 public episodes and 32,212 frames. Its lineage source is only a task/camera-schema proxy, not a physical scene identifier; the corrected proxy split scores 0.516 versus 0.496 for the random split, so the measured score drop is negative. The proof gate therefore records one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured famous-benchmark inflation claims in this release; its command path and acceptance rule are stated in Appendix B. **HELP: C2** The benchmarks lack the public controls needed to rule out leakage; we flag that gap but refuse to call any score inflated without a measured rerun.

The same audit protocol also runs above the episode level: a dataset-scale manifest audit and a generated catalog benchmark (Appendix C) make the DATASET.001 scale claim executable on catalog-side evidence only.

5 Positioning

A contribution of falsifiable invariants. The experiments show several ways a result can be wrong while the files are valid: a split can leak world lineage, replay can drift because control timing is underspecified, conversion can discard physical semantics that the target container has no native place to store, and a live virtual or AV state can drift from its authoritative revision. WorldEpisode is useful only because these failures are observable and testable. The sidecar is the mechanism; the scientific claim is that explicit spatial-state semantics change what an embodied-agent result means. A container adapter either preserves those invariants, externalizes them with declared loss, or fails conformance.

A small semantic core with multiple bindings. The bindings are evaluated at very different depths: LeRobotDataset v3 is the only binding exercised with active real-data experiments (conversion, leakage, replay timing); Rerun, NCore, MCAP/ROS, OpenUSD/SimReady, and glTF/SPZ are covered only as binding-retention pilots (Table 3). OpenUSD standardizes how the 3D world is composed; WorldEpisode standardizes how a robot modifies state within that space over time without silent data corruption. The same invariant structure should generalize to a game-engine or AV profile, though only the robotics profile is implemented and evaluated here.

Real-to-sim pipelines and benchmarks are the users, not the competitors. Gaussian-splat and OpenUSD real-to-sim pipelines are the systems that most need a state and episode contract: a reconstructed room can be excellent visual training data and still be unsafe for policy evaluation if collision roles are missing, lineage leaks across splits, or the replayed action vector is interpreted under the wrong hardware contract. The same stance governs benchmark auditing: when public metadata cannot answer whether train and test share a scene lineage, asset family, source capture, or timing convention, WorldEpisode marks the published score as unaudited with respect to that control — and marks it inflated only after the same policy protocol is rerun under the corrected split or timing contract.

Standards alignment. The project aligns with standards bodies rather than antagonizing them. Static 3D robot-map exchange, humanoid dataset lifecycle, OpenLABEL annotation, glTF Gaussian splats, USD composition, and NCore capture conventions are all useful neighbors. WorldEpisode is the executable profile and bridge across those layers.

6 Limitations and Open Gates

The current evidence is deliberately scoped: it exposes hidden failure mechanisms rather than claiming full robot-learning, game-engine, or autonomous-driving generality. The boundary of every empirical claim is tabulated in Appendix A, and each open gate’s commands and acceptance rules are boxed in Appendix B; this section states what matters most.

Beyond robotics (future work). All measured evidence in this paper is the WorldEpisode robotics profile; the broader Universal Spatial State framing is proposed, not demonstrated outside robotics. Two deterministic pilots only illustrate that the same vocabulary can express non-robotics drift. In a game-engine collision pilot, a client loads a stale arena collision asset after an authoritative patch, which the asset-digest, state-revision, and role checks catch. In an autonomous-driving clock pilot, camera and lidar logs deserialize correctly, but a 50 ms undeclared clock offset at 15 m/s

creates a 0.75 m fusion error against a 0.20 m tolerance, which the declared clock mapping corrects to 0.0 m. These are illustrative pilots, not production game-engine or AV benchmark results. A stronger paper should add at least one public non-robotics corpus and an independently maintained adapter. [HELP: C6](#)

Principal limitations. First, the leakage result is an offline behavioral-cloning audit, not a physical rollout: the Torch MLP probe and the temporal ridge baseline demonstrate that random splits can leak world lineage and inflate offline evaluation, but ACT and Diffusion Policy must be run on the committed split manifest (and the split packages’ source videos mirrored with digests) before claiming impact on common LeRobot training recipes. Second, the reference implementation is single-sourced: the clean-room reader reduces the risk that the fixtures only work through the reference SDK, but it is an internal artifact, and the `worldepisode` package is not yet a released PyPI package or merged into upstream LeRobot or Rerun examples. The pilot natural-source corpus adds 19 cases from five public robot-learning datasets but does not prove prevalence in the wild; a stronger version of this paper should record whether dataset maintainers agree with representative diagnostics. [HELP: C4](#)

7 Conclusion

Robot-learning failures often arrive disguised as clean data: valid tensors, valid videos, valid scene files, and invalid state assumptions. WorldEpisode makes those assumptions explicit. It binds interaction data to immutable state revisions, gives persistent identity to entities, declares space/time/transition semantics, assigns roles to assets, records events and provenance, and reports conversion loss instead of hiding it.

In the reference experiments, those requirements expose a 0.850 offline random-split success rate that collapses to 0.000 under scene-disjoint evaluation, reduce LeRobot replay drift on a real SO-101 trajectory by modeling a four-frame action delay, and catch every injected physically incoherent package before deployment. A real-to-sim pipeline can report simulated success while failing under deployment proxies when action contracts or representation roles drift; the same vocabulary catches that drift in deterministic pilots outside robotics as well. The validator ships as a one-line preflight command, and the simulator pathway is an adapter compliance contract rather than a dependency on one runtime.

These are scoped results, not a completed standardization claim. The call-out audit identifies Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN as priority targets for this protocol, but it does not claim their scores are inflated without reruns; the inflation-proof gate currently records one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured inflation claims. The non-robotics pilots are vocabulary checks, not production game-engine or AV evaluations. Gaussian splats remain a high-value appearance profile, but the durable contribution is the spatial-state contract around them. The next step is to scale the same protocol to larger multi-robot datasets, stronger LeRobot-native policies, real or high-fidelity rollout evaluation, natural failure corpora, non-robotics adapters, and independently maintained bindings.

A Evidence Boundaries

Table 4 states the boundary of each empirical claim in Section 4.

Table 4: What the current evidence supports, and what it does not yet support.

Claim area	Current evidence	Boundary
Leakage	Public ArmnetBench LeRobot audit with 400 teleoperated reference episodes, an executable Torch BC probe, a measured temporal ridge state/action baseline, an ACT/Diffusion gate harness, and four compact digest-verified LeRobot state/action split packages totaling 241,470 frames	Stronger ACT/Diffusion jobs are prepared but not run; the compact split packages omit video payloads, so no vision-policy, real-robot, or high-fidelity rollout result is claimed yet
Conversion	Two pinned public LeRobotDataset v3 five-episode batch round trips with exact tensor, index, and timestamp equality	Two datasets; broader LeRobot coverage remains future work
Replay timing	Real SO-101 trajectory alignment, tested same-trace MuJoCo and Genesis position-servo replay adapters, URDF Studio MuJoCo/Genesis episode-backend evidence, and a dependency-free scheduler conformance harness	One LeRobot trace with minimal joint replay adapters; no contact-rich task rollout, Isaac runtime result, or SAPIEN runtime result is claimed
Meta-simulator neutrality	Adapter contract matrix over MuJoCo, Isaac Sim, Genesis, and SAPIEN plus same-trace MuJoCo/Genesis replay evidence, URDF Studio SimBackend conformance, and a one-episode MuJoCo-Genesis comparison	MuJoCo and Genesis are tested for the minimal replay profile; Isaac and SAPIEN are not replay-tested in this paper, and equal physics is not claimed
Generalization beyond robotics	Deterministic game-engine collision-patch and autonomous-driving clock-domain pilots using the same invariant vocabulary	Not measured on production game engines, public AV datasets, or fleet logs
Validation	Fourteen injected requirement faults, two independent hand-authored fixtures, and a pilot natural-source corpus over five public datasets	Five-dataset count is met through active LeRobot artifacts plus source-level public benchmark metadata; dataset-specific diagnostic reports now cover all cases, but no maintainer feedback is recorded yet
Binding retention	Versioned uss-core-23 semantic projection checked by executable artifacts	Pilot projection; not a universal score of each storage format
Famous benchmark call-out	Source-level audit over Open X-Embodiment, DROID, BridgeData V2, LIBERO, and CALVIN, plus a targeted DROID subset rerun tool and a separate inflation-proof gate	Missing public controls are not measured score inflation; the proof gate currently has one executed DROID subset rerun, zero inflation-proof famous-benchmark rerun reports, and zero measured inflation claims
Real-to-sim drift	Controlled action-contract and representation-role ablations where drifted contracts succeed in simulation and fail under deployment proxies	Deterministic proxy; not a RoboSnap/DROID-Sim rerun or physical hardware deployment
Adoption	Public schema, installable local preflight package, validator, fixtures, governance files, and an internal clean-room reader that does not import the reference SDK	No PyPI release, upstream LeRobot/Rerun integration, external independent implementation, or external dataset release yet
Scale architecture	Dataset manifest schema, scalable example, executable audit, and a generated 32,768-shard catalog benchmark describing 1,073,741,824 episodes for open/index latency, partition pruning, digest-cache behavior, and resolver routing	Catalog-side benchmark only; no billion episode rows, payload bytes, distributed storage, concurrent readers, or institutional federation are measured

B Open Gates: Commands and Acceptance Rules

Open result, not claimed: results not claimed in this release

The artifact keeps unfinished claims executable instead of hiding them. The current RFC release gate is: `python3 tools/release_readiness.py -strict-rfc`. The full-standard gate is: `python3 tools/release_readiness.py -strict-full`. The latter is expected to fail until the two gates boxed below are backed by committed reports and, additionally, external standard evidence exists: maintainer-reviewed diagnostics, an independently written reader/exporter, and at least one externally published WorldEpisode-compatible dataset. The same gates are indexed in `docs/experiments/open_reproduction_gates/open_reproduction_gates.json`.

Open result, not claimed: ACT/Diffusion and rollout impact

Status: open (referenced from Case 1). To close this gate, first regenerate the split packages and LeRobot job specs with `python3 tools/lerobot_policy_leakage_gate.py`. Then run the generated jobs with `bash docs/experiments/lerobot_policy_gate/run_lerobot_policy_jobs.sh` inside a LeRobot training environment.

Required artifacts: policy checkpoint digests, train/eval configs and seeds, offline action metrics for random and scene-disjoint splits, rollout traces or videos with content digests, and an updated `docs/experiments/lerobot_policy_gate/policy_gate_report.json`.

Acceptance rule: at least one ACT or Diffusion Policy run must report both split protocols, and at least one simulator or physical rollout report must use the same split manifest. Until then, the paper claims only the offline Torch probe and the prepared gate.

Open result, not claimed: famous-benchmark score-inflation proof

Status: open (referenced from Case 7). A targeted DROID subset rerun can be attempted with `uv run -with pyarrow -with requests -with numpy python tools/famous_benchmark_policy_rerun.py -benchmark droid_100 -required`. The proof contract is then enforced with `python3 tools/benchmark_inflation_gate.py -required`.

Required artifacts: benchmark-specific WorldEpisode conversion, lineage/timing audit, faithful published-protocol policy rerun, paired corrected evaluation under the same metric, and a measured baseline-minus-corrected score drop.

Acceptance rule: the gate must contain at least one inflation-proof valid rerun report with `measured_inflation=true`. Source-level metadata gaps alone are evidence of missing controls, not evidence that published scores are inflated.

C Dataset-Scale Audit (DATASET.001)

To address production scale, we audit the dataset manifest in `examples/scalable-corpus.worldepisode-dataset.json`. The audit validates five namespaces, three resolver schemes, three registries, three shards, two indexes, one append-only version, and nine digest-addressed asset descriptors spanning Hugging Face, S3, and OCI URIs with local mirrors. It also checks that the manifest includes a world-lineage index, an asset-digest index, and a split-manifest shard. We then run a generated catalog benchmark with 32,768 trace shards describing 1,073,741,824 episodes. The committed report records the measured time for parsing and indexing the generated JSON catalog, eight partition-pruning queries, digest-cache resolution, and resolver routing over all asset descriptors. This is catalog-side evidence only, but it makes the scale claim executable: opening a corpus starts from the manifest and indexes rather than from recursive storage discovery.

D Open Contributions (colleague working notes)

This appendix is a coordination surface, not a paper result. It renders only in the working build (the `\collabtrue` switch in `main.tex`); flip that switch to `\collabfalse` to remove it before any external submission. Each card below is a claimable unit of work with a fixed anchor: **Where** it lives in this paper and the repo, **What is missing**, **How to close it** (exact commands), and **Needs** (environment and access). The purple `HELP: Cx` tags in the Evidence and Limitations sections point back to these cards.

How to claim a card. Put your name in the **Owner** slot, open an RFC issue for any normative change (per `GOVERNANCE.md`), and keep published requirement IDs stable. The four experiment gates (C1–C4) are the same ones indexed in `docs/experiments/open_reproduction_gates/open_reproduction_gates.json`.

Finishing a card also means finishing the prose. The paper text is complete but evidence-gated: closing a card is not done until you (1) fold the measured numbers into the cited section, (2) update the evidence-boundary table in the Limitations section and the summary figure in the Failure Case Studies section, and (3) keep the gates green by re-running `python3 tools/paper_claim_audit.py --strict` and `python3 tools/artifact_freshness.py --strict`. The whole readiness suite is `make readiness`.

Experiment gates

[C1] ACT/Diffusion policy metrics and rollout impact

Where: Evidence, Case 1 (scene-lineage leakage); Appendix B, “ACT/Diffusion and rollout impact” box; artifacts under `docs/experiments/lerobot_policy_gate/`. Gate `POLICY.ROLL.001`.

What is missing: ACT and Diffusion Policy offline metrics on *both* the random and scene-disjoint splits, plus at least one simulator or physical rollout on the same split manifest. Source videos must be mirrored with digests before any vision-policy claim; the current gate is `ready_not_executed`.

How to close it: `python3 tools/lerobot_policy_leakage_gate.py`, then `bash docs/experiments/lerobot_policy_gate/run_lerobot_policy_jobs.sh` in a LeRobot training environment. The compact physical split packages under `docs/experiments/lerobot_policy_gate/physical_splits/` are already committed, so offline training needs no download; only source videos must be mirrored (with digests) for a vision policy. Commit the policy metrics, rollout traces or videos with digests, and the updated `docs/experiments/lerobot_policy_gate/policy_gate_report.json`.

Needs: GPU plus a LeRobot / ACT / Diffusion Policy training environment; Hugging Face access to the source videos.

Owner: _____ **Target:** _____

[C2] Famous-benchmark score-inflation proof

Where: Evidence, Case 7 (famous benchmarks); Appendix B, “famous-benchmark score-inflation proof” box. Gate `BENCH.INFLATE.001`.

What is missing: A benchmark-specific WorldEpisode conversion, a faithful published-protocol policy rerun, and a paired corrected evaluation showing a measured baseline-minus-corrected score drop. The committed DROID-100 attempt is fail-closed (Hugging Face DNS resolution failed) and its task/camera proxy split shows no drop, so zero inflation-proof reports exist yet.

How to close it: `uv run --with pyarrow --with requests --with numpy python tools/famous_benchmark_policy_rerun.py --benchmark droid_100 --required`, then `python3 tools/benchmark_inflation_gate.py --required`. Offline: pre-place the pinned Parquet shards under `--cache-root <dir>` at the repo-id/revision path the tool prints, then re-run with that flag to skip all network access (the runner also attempts a DNS-over-HTTPS fallback automatically).

Needs: Reliable network access to Hugging Face; the published policy checkpoint and evaluation protocol for the target benchmark; compute for the rerun.

Owner: _____ **Target:** _____

[C3] Maintainer-confirmed natural-failure prevalence

Where: Evidence, Case 4 (the validator) and its natural-source corpus; Limitations, “Principal limitations”. Gate: natural-failure prevalence.

What is missing: Pinned WorldEpisode conversions for the source-level datasets (DROID, BridgeData V2), false-positive review of the emitted diagnostics, and recorded agreement from the dataset maintainers. Today two cases are source-level metadata only.

How to close it: Extend the converters to pin those datasets, re-run `python3 tools/run_experiments.py`, then contact the dataset maintainers with representative diagnostics and record their responses.

Needs: Dataset domain knowledge; maintainer contacts; storage for the pinned shards.

Owner: _____ **Target:** _____

[C4] Mature external-standard adoption

Where: Limitations, “Principal limitations” and the Adoption row of the evidence-boundary table (Appendix A); `GOVERNANCE.md` “Independence Target”. Gate: external adoption.

What is missing: A second, independently written reader/exporter (not the reference SDK), at least one externally published WorldEpisode-compatible dataset, a PyPI release of the `worldepisode` package, and upstream LeRobot / Rerun integration.

How to close it: Build the independent implementation and validate it with `python3 tools/cleanroom_conformance_reader.py`; publish the wheel to PyPI; open the upstream pull requests; re-check `python3 tools/release_readiness.py --strict-rfc`.

Needs: A second implementer, ideally at another institution (the independence requirement); packaging and release access.

Owner: _____ **Target:** _____

Standalone technical items

[C5] Isaac Sim / SAPIEN replay and a contact-rich rollout

Where: Evidence, Case 2 (action-timing and replay drift); “Replay timing” and “Meta-simulator neutrality” rows of the evidence-boundary table (Appendix A).

What is missing: Tested Isaac Sim and SAPIEN replay adapters (Isaac is adapter-ready but untested; SAPIEN is adapter-required) and at least one contact-rich task rollout. Only MuJoCo and Genesis same-trace replay is measured today.

How to close it: Implement the adapters against the meta-simulator contract, validate scheduling with `python3 tools/replay_adapter_conformance.py`, then run real Isaac / SAPIEN same-trace replays and commit the RMSE reports.

Needs: IsaacLab and SAPIEN installations; GPU.

Owner: _____ **Target:** _____

[C6] A public non-robotics corpus (generalization beyond robotics)

Where: Limitations, “Beyond robotics (future work)”.

What is missing: At least one public non-robotics corpus (game-engine or autonomous-driving) bound through an independently maintained adapter, beyond the two deterministic pilots.

How to close it: Pick a public AV or game-telemetry dataset, write a binding plus a conversion-loss report, add conformance fixtures, and wire it into `tools/run_experiments.py`.

Needs: Access to a public AV or game-telemetry dataset; a maintainer for that adapter.

Owner: _____ **Target:** _____

[C7] Broader conversion coverage

Where: Evidence, Case 5 (loss-explicit conversion); Conversion row of the evidence-boundary table (Appendix A).

What is missing: More than two round-tripped LeRobot datasets, and at least one larger multi-camera manipulation dataset.

How to close it: `python3 tools/lerobot_worldepisode_roundtrip.py --required --repo-id <dataset> --revision <sha> --batch-episode-indices 0,1,2,3,4`; commit the round-trip reports.

Needs: Hugging Face access; disk for the shards.

Owner: _____ **Target:** _____

References

- [1] Alliance for OpenUSD. OpenUSD: Universal scene description. <https://openusd.org/>, 2026. Accessed 2026-07-13.
- [2] Armnet. ArmnetBench v0.1: LeRobot single-arm SO-101 release. https://huggingface.co/datasets/armnet/armnetbench_v01_lerobot_so101, 2026. Pinned revision 2e5e89aee0e7f081078d9d6ab3b279fc4b83ea84; accessed 2026-07-13.
- [3] Remi Cadene, Simon Aliberts, Francesco Capuano, Michel Aractingi, Adil Zouitine, Pepijn Kooijmans, Jade Choghari, Martino Russi, Caroline Pascal, Steven Palma, Mustafa Shukor, Jess Moss, Alexander Soare, Dana Aubakirova, Quentin Lhoest, Quentin Gallouedec, and Thomas Wolf. LeRobot: An open-source library for end-to-end robot learning. In *International Conference on Learning Representations*, 2026. arXiv:2602.22818.
- [4] CALVIN Authors. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. <https://calvin.cs.uni-freiburg.de/>, 2021. Accessed 2026-07-13.

- [5] DROID Dataset Authors. DROID: A large-scale in-the-wild robot manipulation dataset. <https://droid-dataset.github.io/>, 2024. Accessed 2026-07-13.
- [6] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.
- [7] Google DeepMind. MuJoCo: Multi-joint dynamics with contact. <https://mujoco.org/>, 2026. Accessed 2026-07-13.
- [8] Hugging Face. LeRobotDataset v3.0. <https://huggingface.co/docs/lerobot/lerobot-dataset-v3>, 2026. Accessed 2026-07-13.
- [9] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.
- [10] LeRobot. pusht: A public LeRobotDataset v3 dataset. <https://huggingface.co/datasets/lerobot/pusht>, 2025. Pinned revision 7628202a2180972f291ba1bc6723834921e72c19; accessed 2026-07-13.
- [11] LeRobot. svla_so101_pickplace: A public LeRobotDataset v3 dataset. https://huggingface.co/datasets/lerobot/svla_so101_pickplace, 2025. Pinned revision f641879e22172be7e8161d5e6c1503c2d2feb657; accessed 2026-07-13.
- [12] LIBERO Authors. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. <https://libero-project.github.io/main.html>, 2023. Accessed 2026-07-13.
- [13] MLCommons. Croissant: A metadata format for ML-ready datasets. <https://mlcommons.org/croissant/>, 2024. Accessed 2026-07-13.
- [14] NVIDIA. NCore Specification: Data Conventions. <https://nvidia.github.io/ncore/data/conventions>, 2026. Accessed 2026-07-13.
- [15] Open Robotics. ROS REP 103: Standard units of measure and coordinate conventions. <https://www.ros.org/reps/rep-0103.html>, 2026. Accessed 2026-07-13.
- [16] Open Robotics. URDF: Unified robot description format. <https://wiki.ros.org/urdf>, 2026. Accessed 2026-07-13.
- [17] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and rt-x models. <https://robotics-transformer-x.github.io/>, 2023. Accessed 2026-07-13.
- [18] Rerun. A new data layer for robot learning. <https://rerun.io/blog/data-layer-for-robot-learning>, 2026. Accessed 2026-07-13.
- [19] RoboSnap Authors. RoboSnap: One-Shot Real-to-Sim Scene Generation for Generalizable Robot Learning and Evaluation. <https://arxiv.org/html/2607.06699v1>, 2026. Accessed 2026-07-13.
- [20] The Khronos Group. KHR_gaussian_splatting. https://github.com/KhronosGroup/glTF/blob/main/extensions/2.0/Khronos/KHR_gaussian_splatting/README.md, 2026. Accessed 2026-07-13.

- [21] The Khronos Group. Khronos Announces glTF Gaussian Splatting Extension. <https://www.khronos.org/news/press/glTF-gaussian-splatting-press-release>, 2026. Accessed 2026-07-13.
- [22] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Quan Vuong, Philippe Hansen-Estruch, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. BridgeData V2: A dataset for robot learning at scale. In *Conference on Robot Learning*, 2023. Accessed 2026-07-13.